

Contents

Chapter 1

AI Prompt Engineering

For the technical accuracy of the output, following the guideline of to the project guidelines, the report is generated using **Two-phase prompt strategy**: prompt improvement and prompt execution.

1.1 Prompt Engineering Phases

1.1.1 Prompt Improvement

The first phase is the prompt improvement, where the initial prompt is redefined and improved the given prompt following the project guidelines and a rigorous academic approach. Every prompt improvement is independent from the previous prompt, and the output is generated using the improved prompt.

1.1.2 Prompt Execution

The second phase is the prompt execution, where the improved prompt is used to generate the final output. In this phase, the chat is invited to also give the source reference used to generate the output.

Chapter 2

Prompts

2.1 Prompt 0: configure prompt engineering

Model: Gemini 3.1 Pro extended reasoning

Prompt:

System Persona: Act as a Senior Prompt Engineer specializing in constructing technical prompts for academic writing and code-generation tasks.

Goal: Given, as input, a task specification (persona, goal, requirements, writing-style constraints, output structure) for producing a technical-academic report, produce a single, self-contained master prompt -- ready to be pasted into a new conversation with a language model -- which, when executed, will generate that report.

Requirements:

Task 1: Extraction & Preservation. Parse the input specification and identify its persona/role, goal (including title), all requested tasks with their respective details, writing-style constraints, and required output structure. Preserve the full content and intent of the original specification: do not add requirements, do not omit any, and do not alter their meaning.

Task 2: Correction & Structuring. Correct any ambiguities, grammatical errors, or malformed formatting present in the input (e.g., improperly nested quotation marks). Reorganize the extracted elements into a clear, logically structured master prompt, using the same section labels as the input where applicable (e.g., Persona, Goal, Requirements, Writing Style Constraint, Output Structure).

Writing Style Constraint (STRICT): Do not execute the input specification and do not produce the report itself. Do not include

placeholders, editorial notes, comments, or meta-commentary addressed to the user. Do not use first-person forms ("I," "we") anywhere in the generated master prompt unless they were already required by the input specification's own writing-style constraints.

Output Structure: Return only the generated master prompt, formatted as ready-to-use text, structured under the same section headings as the input (System Persona / Goal / Requirements / Writing Style Constraint / Output Structure), with no additional text before or after it.

Output: Add to the context of the model the persona, formatting constraints and the goal of the report.

2.2 Prompt 1: overview of the Parametric Polymorphism

Model: Gemini 3.1 Pro extended reasoning.

Goal: Generate an overview of the Parametric Polymorphism and ad-hoc polymorphism. Giving details of type erasure, compared with generics, and monomorphization. Also compared differences between source code, byte code and binary code.

Prompt:

- Provide an exhaustive theoretical background on parametric polymorphism.
- Define and thoroughly explain Christopher Strachey's distinction between parametric polymorphism and ad-hoc polymorphism.
- Detail the two primary implementation strategies discussed in the literature:
 1. Homogeneous translation (type erasure), detailing how it results in one compiled representation for all instantiations.
 2. Heterogeneous translation (reification/monomorphization), detailing how it results in separate code generated per instantiated type.
- Analyze type erasure versus reified generics as the core axis of comparison.
- Analyze compile-time versus runtime characteristics as the second axis of comparison, specifically examining when instantiation happens and what type information survives into the compiled binary or bytecode.
- The entire document must be written in professional, compilation-ready LaTeX code.

Resources: The generation of the technical report was supported by the following academic and technical references. The selected resources provide the theoretical foundations of parametric polymorphism, the classification of polymorphic systems, and the implementation techniques adopted by modern programming languages.

The theoretical background was derived from foundational works on type systems and polymorphism, including Strachey's original classification of polymorphism, Cardelli and Wegner's analysis of type abstraction and polymorphism, and Pierce's formal treatment of parametric polymorphism and type systems [?, ?, ?, ?].

The analysis of generic implementation strategies, including homogeneous translation, type erasure, heterogeneous translation, reification, and monomorphization,

was based on research concerning generic programming implementations and runtime systems, including the .NET Common Language Runtime and Java generic mechanisms [?, ?, ?, ?].

The comparison between compile-time and runtime characteristics, including instantiation time, runtime representation, reflection capabilities, runtime type information, optimization opportunities, performance trade-offs, and type safety guarantees, was supported by compiler literature and modern programming language implementation documentation [?, ?, ?, ?, ?, ?, ?].

Bibliography

- [1] Alfred V. Aho, Monica S. Lam, Ravi Sethi, and Jeffrey D. Ullman. *Compilers: Principles, Techniques, and Tools*. Pearson, 3 edition, 2022.
- [2] Andrew W. Appel. *Modern Compiler Implementation*. Cambridge University Press, 2 edition, 2022.
- [3] Gilad Bracha. Generics in the java programming language, 2004.
- [4] Luca Cardelli and Peter Wegner. On understanding types, data abstraction, and polymorphism. *ACM Computing Surveys*, 17(4):471–523, 1985.
- [5] Maurizio Cimadamore and Brian Goetz. State of valhalla: Specialization and generics in the java virtual machine, 2020.
- [6] Keith D. Cooper and Linda Torczon. *Engineering a Compiler*. Morgan Kaufmann, 3 edition, 2022.
- [7] The Rust Project Developers. Monomorphization and code generation in rust, 2024.
- [8] The Rust Project Developers. The rust reference, 2024. Official Rust language specification.
- [9] Ecma International. Ecma-335: Common language infrastructure (cli), 2024.
- [10] Andrew Kennedy and Don Syme. Design and implementation of generics for the .net common language runtime. In *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, pages 1–12. ACM, 2001.
- [11] John McCall. Generics in swift: Implementation and runtime representation, 2019.
- [12] Benjamin C. Pierce. *Types and Programming Languages*. MIT Press, Cambridge, MA, 2002.
- [13] Benjamin C. Pierce. *Advanced Topics in Types and Programming Languages*. MIT Press, Cambridge, MA, 2009.
- [14] Christopher Strachey. Fundamental concepts in programming languages, 1967. Lecture notes, International Summer School in Computer Programming, Copenhagen.

- [15] David Vandevor, Nicolai M. Josuttis, and Douglas Gregor. *C++ Templates: The Complete Guide*. Addison-Wesley Professional, 2 edition, 2017.